

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Laboratorio de Microcomputadoras

Práctica 06 Convertidor Analógico Digital, Control PWM

Grupo de laboratorio: 02

Profesor: Ing. Moisés Meléndez Reyes

Brigada: 04

Integrantes:

- Hernández Acosta Mauricio Gabriel 321121051
- Pérez Olvera Alexis Abraham 321150503
- Sierra García Mariana 320301238



Fecha de entrega: 18 de octubre del 2026

Semestre: 2027-1

Índice

1. Objetivo **2**

2. Actividades **2**
2.1. Ejercicio 1 . . . 2
2.1.1. Pseudo Código . . . 2
2.1.2. Desarrollo . . . 2
2.1.3. Análisis de resultados . . . 2
2.2. Ejercicio 2 . . . 3
2.2.1. Pseudo Código . . . 3
2.2.2. Desarrollo . . . 4
2.2.3. Análisis de resultados . . . 4
2.3. Ejercicio 3 . . . 5
2.3.1. Pseudo Código . . . 5
2.3.2. Desarrollo . . . 5
2.3.3. Análisis de resultados . . . 6
2.4. Ejercicio 4 . . . 7
2.4.1. Pseudo Código . . . 7
2.4.2. Desarrollo . . . 7
2.4.3. Análisis de resultados . . . 8
2.5. Ejercicio 5 . . . 9
2.5.1. Pseudo Código . . . 9
2.5.2. Desarrollo . . . 9
2.5.3. Análisis de resultados . . . 10
2.6. Ejercicio 6 . . . 11
2.6.1. Pseudo Código . . . 11
2.6.2. Desarrollo . . . 11
2.6.3. Análisis de resultados . . . 12
2.7. Ejercicio 7 . . . 13
2.7.1. Pseudo Código . . . 13
2.7.2. Desarrollo . . . 13
2.7.3. Análisis de resultados . . . 14

3. Conclusiones **15**
3.1. Hernandez Acosta Mauricio Gabriel . . . 15
3.2. Perez Olvera Alexis Abraham . . . 15
3.3. Sierra Garcia Mariana . . . 15

1. Objetivo

Conocer el funcionamiento del ADC (convertidor analógico digital) para realizar aplicaciones que requieran procesar señales en el mundo continuo, convertir señales provenientes de sensores; aprender el funcionamiento del control PWM.

2. Actividades

2.1. Ejercicio 1

Comentar el siguiente programa, e indicar que hace:

```
from machine import ADC, Pin
import time

Sensor = ADC(4)

while True:

    Valor = Sensor.read_u16()*(3.3/65535)
    Temp = 27-(Valor-0.706)/0.001721
    print(Temp)
```

2.1.1. Pseudo Código

Algoritmo 1: Lectura de temperatura del sensor interno

Inicializar ADC en el canal 4;
Mientras el programa esté en ejecución hacer;
    Leer el valor analógico del sensor;
    Convertir el valor a voltaje;
    Calcular la temperatura usando la fórmula del sensor interno;
    Mostrar la temperatura en consola;

2.1.2. Desarrollo

```
from machine import ADC, Pin
import time

Sensor = ADC(4)  # ADC4 es el sensor de temperatura interno

while True:

    Valor = Sensor.read_u16()*(3.3/65535)  # Conversión a voltaje
    Temp = 27-(Valor-0.706)/0.001721       # Fórmula del sensor interno
    print(Temp)                            # Mostrar temperatura
    time.sleep(1)
```

2.1.3. Análisis de resultados

El programa realiza la lectura del sensor de temperatura interno del microcontrolador mediante el ADC. Convierte la lectura analógica a voltaje y posteriormente a temperatura utilizando la fórmula del fabricante. El valor de temperatura se muestra continuamente en la consola de Thonny cada segundo, permitiendo monitorear la temperatura del sistema en tiempo real.



Figura 1

2.2. Ejercicio 2

Realizar un programa que despliegue en la consola de Thonny el resultado de la conversión y el voltaje generado por el potenciómetro; actualizar cada segundo.

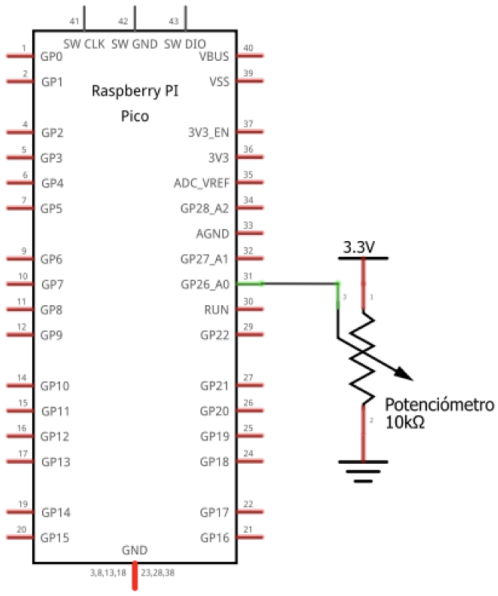




Figura 2: Circuito y ejecución; actividad 2

2.2.1. Pseudo Código

Algoritmo 2: Lectura de potenciómetro

Inicializar ADC en GPIO26;
Mientras el programa esté en ejecución hacer;
    Leer valor analógico;
    Convertir a voltaje;
    Mostrar valor y voltaje en consola;
    Esperar 1 segundo;

2.2.2. Desarrollo

```
from machine import ADC
import time

pot = ADC(26)

while True:
    valor = pot.read_u16()
    voltaje = valor * (3.3 / 65535)

    print("Valor ADC:", valor)
    print("Voltaje:", voltaje)
    print("------------------")
    time.sleep(1)
```

2.2.3. Análisis de resultados

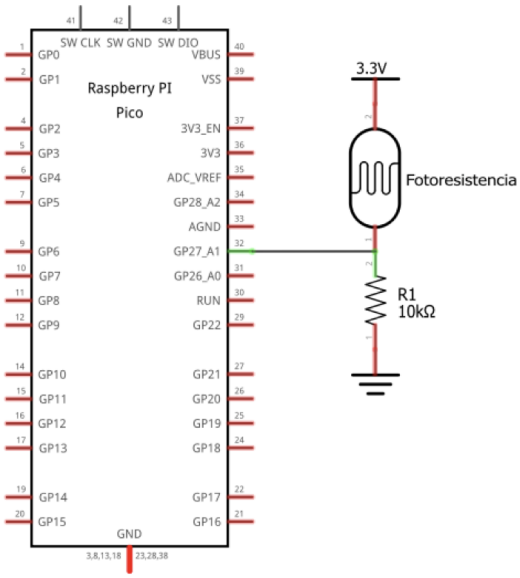
El programa permite leer el valor analógico generado por el potenciómetro mediante el ADC del microcontrolador. Se convierte la señal a voltaje y se muestra en la consola cada segundo. Al girar el potenciómetro se observa el cambio del voltaje, lo que confirma el correcto funcionamiento del divisor de voltaje.



Figura 3

2.3. Ejercicio 3

Empleando el divisor de voltaje generado por la foto-resistencia; tomar la lectura de voltaje, registrar el máximo y mínimo obtenido, fijar un valor de referencia. Mientras la lectura es menor a la referencia, mantenga en bajo el GPIO7 y cuando sobrepase el valor configurado lo ponga en alto.



Lectura de la foto-resistencia:

V_{MIN} =

V_{MAX} =

V_{REF} =

$V_{Foto_resistencia} < V_{REF}$	$V_{Foto_resistencia} > V_{REF}$
GPIO7 = '0'	GPIO7 = '1'

Figura 4: Circuito y ejecución; actividad 2

2.3.1. Pseudo Código

Algoritmo 3: Control GPIO con fotoresistencia

Inicializar ADC en GPIO26;
Inicializar GPIO7 como salida;
Definir valor de referencia;
Mientras el programa esté en ejecución hacer;
    Leer voltaje de la fotoresistencia;
    Si el voltaje es menor a la referencia;
        GPIO7 = 0;
    Si no;
        GPIO7 = 1;

2.3.2. Desarrollo

```
from machine import ADC, Pin
import time

ldr = ADC(26)
gpio7 = Pin(7, Pin.OUT)

referencia = 1.5

while True:

    valor = ldr.read_u16()
    voltaje = valor * (3.3 / 65535)
```

```
13
14      if voltaje < referencia:
15          gpio7.value(0)
16      else:
17          gpio7.value(1)
18
19      print(voltaje)
20
21      time.sleep(1)
```

2.3.3. Análisis de resultados

El sistema utiliza una fotoresistencia para detectar cambios de luz. Cuando la iluminación es baja, el voltaje disminuye y el GPIO7 se mantiene en bajo. Cuando la luz aumenta y supera el valor de referencia, el GPIO7 se activa. Esto permite implementar un sistema de detección de luz o encendido automático.

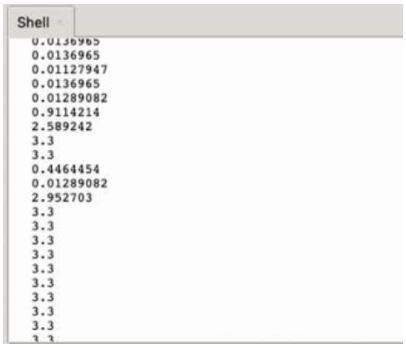


Figura 5

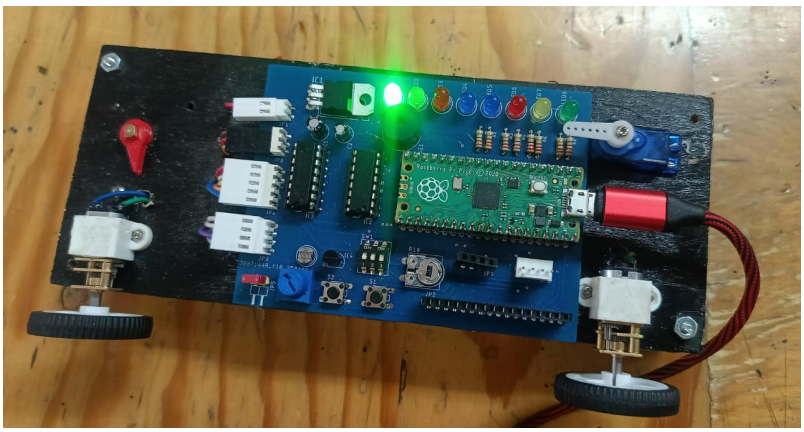
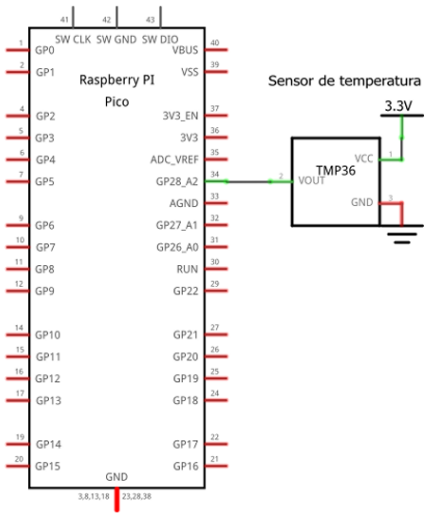


Figura 6

2.4. Ejercicio 4

Realizar un programa que muestre el valor de temperatura tanto en oC como oF, en la consola de Thonny del sensor interno y del sensor TMP36; indicando cual de ellos tiene valor mayor al otro.




```
Temperatura interna = 19ºC; 66ºF
Temperatura externa TMP36 = 20ºC; 68ºF
El sensor TMP36 tiene el valor mayor de temperatura
```

2.4.1. Pseudo Código

Algoritmo 4: Comparación de sensores de temperatura

Inicializar ADC4 sensor interno;
Inicializar ADC26 TMP36;
Mientras el programa esté en ejecución hacer;
    Leer temperatura interna;
    Leer temperatura TMP36;
    Convertir a Celsius;
    Convertir a Fahrenheit;
    Comparar temperaturas;
    Mostrar resultados;

2.4.2. Desarrollo

```
from machine import ADC
import time

sensor_int = ADC(4)
tmp36 = ADC(26)

while True:

    # Sensor interno
    v1 = sensor_int.read_u16() * (3.3 / 65535)
    t1 = 27 - (v1 - 0.706) / 0.001721

    # TMP36
    v2 = tmp36.read_u16() * (3.3 / 65535)
    t2 = (v2 - 0.5) * 100

```

```
17      f1 = (t1 * 9/5) + 32
18      f2 = (t2 * 9/5) + 32
19
20      print("Sensor interno:", t1, "C", f1, "F")
21      print("TMP36:", t2, "C", f2, "F")
22
23      if t1 > t2:
24          print("Sensor interno mayor")
25      else:
26          print("TMP36 mayor")
27
28      print("----------------")
29
30      time.sleep(1)
```

2.4.3. Análisis de resultados

El programa mide la temperatura usando dos sensores distintos. Se convierten los valores a grados Celsius y Fahrenheit y se comparan los resultados. Generalmente el TMP36 ofrece una medición más estable que el sensor interno, ya que el sensor interno puede variar por el calentamiento del microcontrolador.



Figura 7

2.5. Ejercicio 5

Escribir el siguiente programa, comentar e indicar que hace:

```
from machine import Pin, PWM
import time
pwm=PWM(Pin(1))
pwm.freq(1000)

while True:
    for duty in range(0, 65535, 500):
        pwm.duty_u16(duty)
        time.sleep(0.05)
    pwm.duty_u16(0)
    time.sleep(2)
```

2.5.1. Pseudo Código

Algoritmo 5: Generación de PWM

Inicializar PWM en GPIO1;
Configurar frecuencia 1000 Hz;
Mientras el programa esté en ejecución hacer;
    Incrementar duty cycle;
    Aplicar PWM;
    Esperar;
    Apagar PWM;
    Esperar 2 segundos;

2.5.2. Desarrollo

```
from machine import Pin, PWM
import time

pwm = PWM(Pin(1))
pwm.freq(1000)

while True:

    for duty in range(0, 65535, 500):
        pwm.duty_u16(duty)
        time.sleep(0.05)

    pwm.duty_u16(0)
    time.sleep(2)
```

2.5.3. Análisis de resultados

El programa genera una señal PWM que incrementa gradualmente su ciclo de trabajo. Esto permite controlar la intensidad de un LED o la velocidad de un motor. Después de alcanzar el máximo, el PWM se apaga y espera dos segundos antes de reiniciar el ciclo.

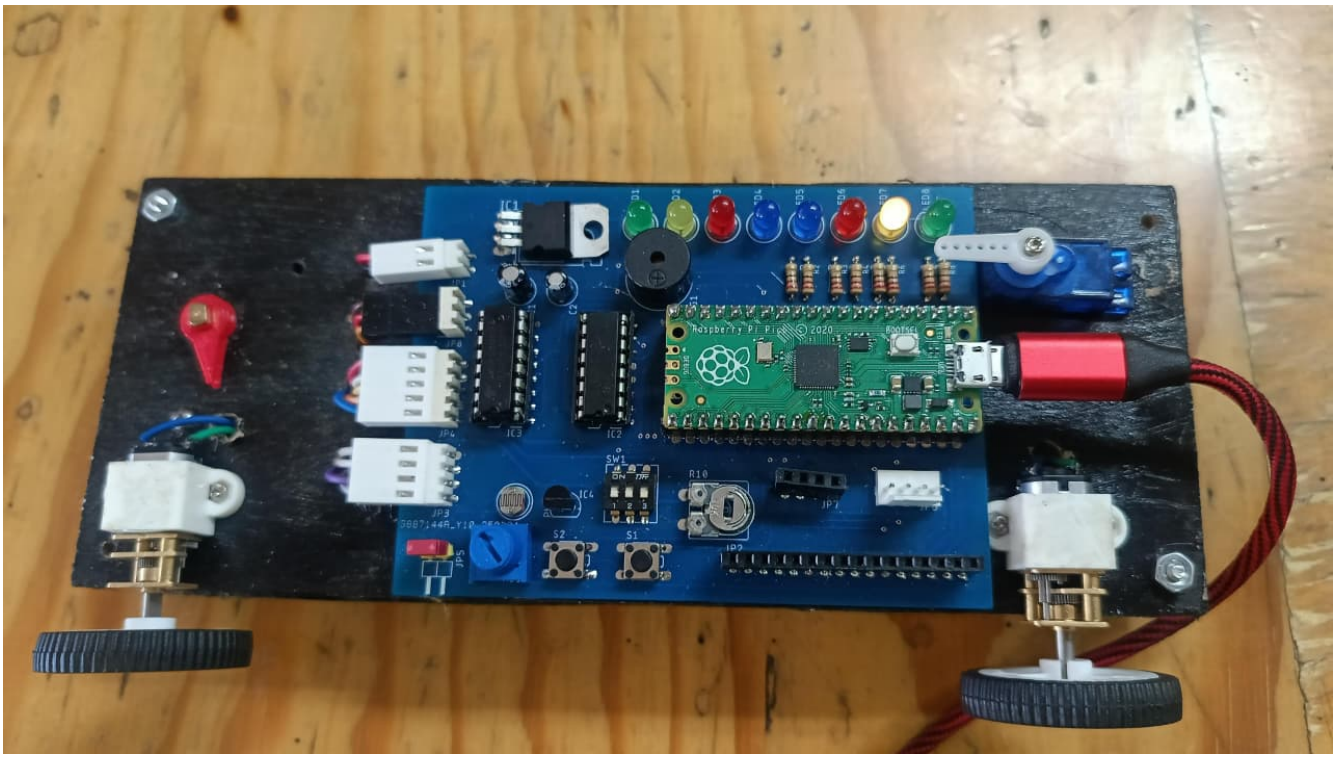


Figura 8

2.6. Ejercicio 6

Modificar el programa para que incremente y decremente de manera automática el valor de PWM en la terminal GPIO0 y refleje el valor inverso en el pin GPIO1.

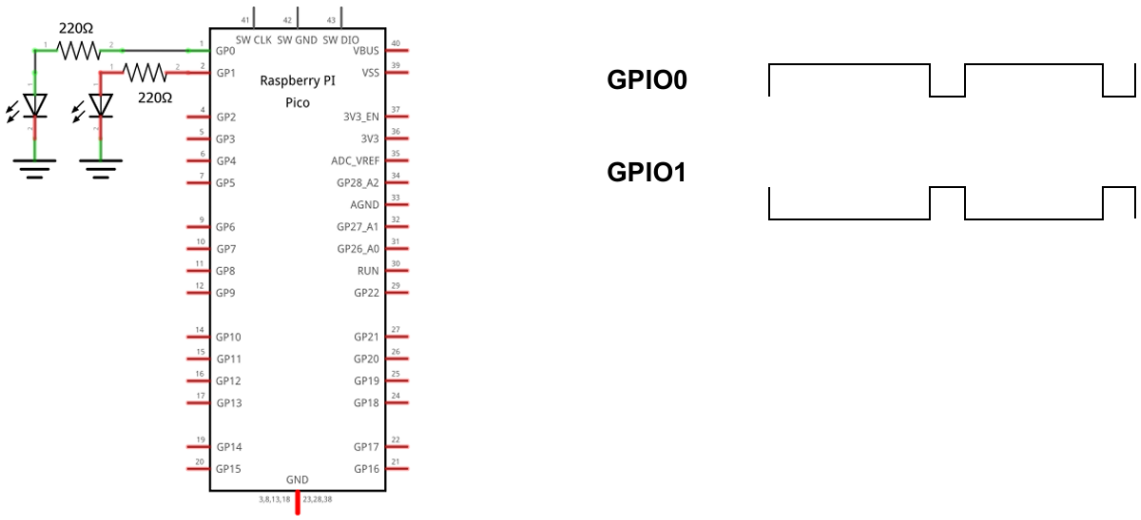


Figura 9: Circuito y oscilograma requerido; actividad 6

2.6.1. Pseudo Código

Algoritmo 6: PWM inverso en GPIO0 y GPIO1

Inicializar GPIO0 como PWM;
Inicializar GPIO1 como PWM;
Configurar frecuencia de 1000 Hz en ambos pines;
Mientras el programa esté en ejecución hacer;
    Para duty desde 0 hasta 65535 hacer;
        PWM GPIO0 = duty;
        PWM GPIO1 = 65535 - duty;
        Esperar 0.05 segundos;
    Para duty desde 65535 hasta 0 hacer;
        PWM GPIO0 = duty;
        PWM GPIO1 = 65535 - duty;
        Esperar 0.05 segundos;

2.6.2. Desarrollo

```
from machine import Pin, PWM
import time

pwm0 = PWM(Pin(0))
pwm1 = PWM(Pin(1))

pwm0.freq(1000)
pwm1.freq(1000)

while True:
```

```
11
12      for duty in range(0, 65535, 500):
13
14          pwm0.duty_u16(duty)
15          pwm1.duty_u16(65535 - duty)
16
17          time.sleep(0.05)
18
19      for duty in range(65535, 0, -500):
20
21          pwm0.duty_u16(duty)
22          pwm1.duty_u16(65535 - duty)
23
24          time.sleep(0.05)
```

2.6.3. Análisis de resultados

El PWM de GPIO0 incrementa mientras el GPIO1 decrece. Esto genera señales inversas, como se observa en los leds.

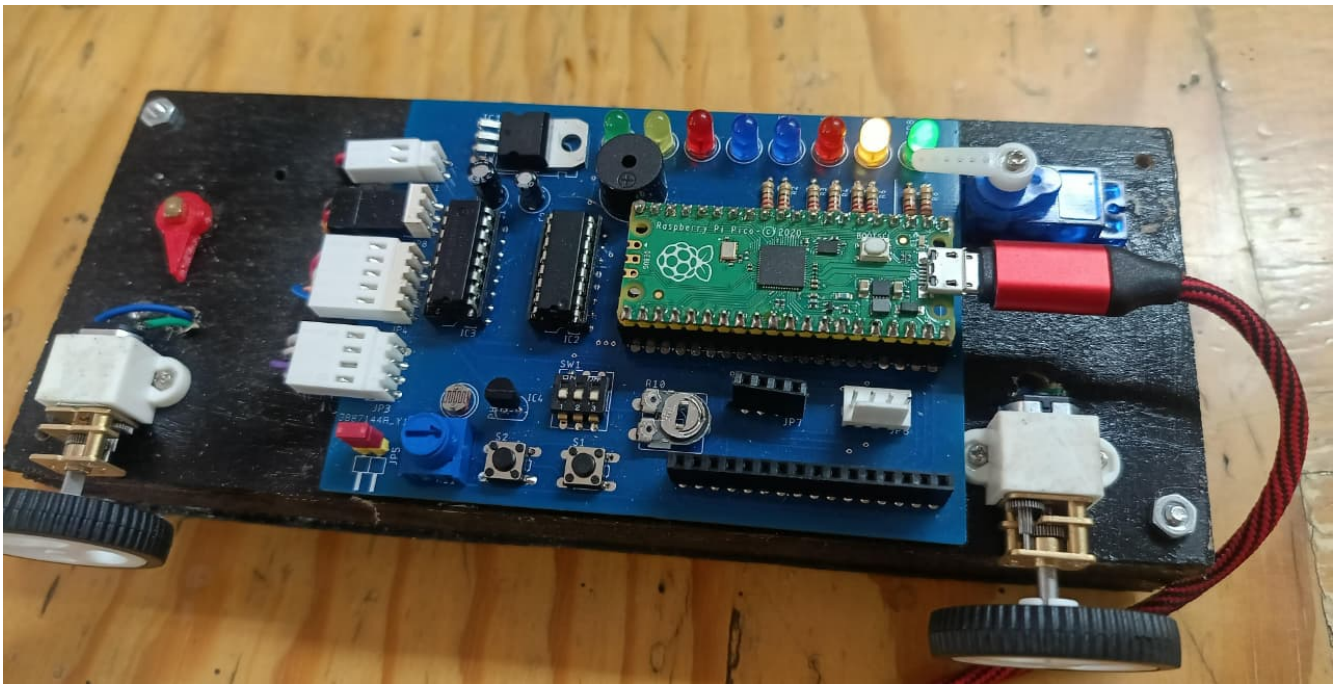


Figura 10

2.7. Ejercicio 7

Realizar las modificaciones necesarias para realizar el control anterior pero ahora a través de la selal analógica proveniente del potenciómetro.

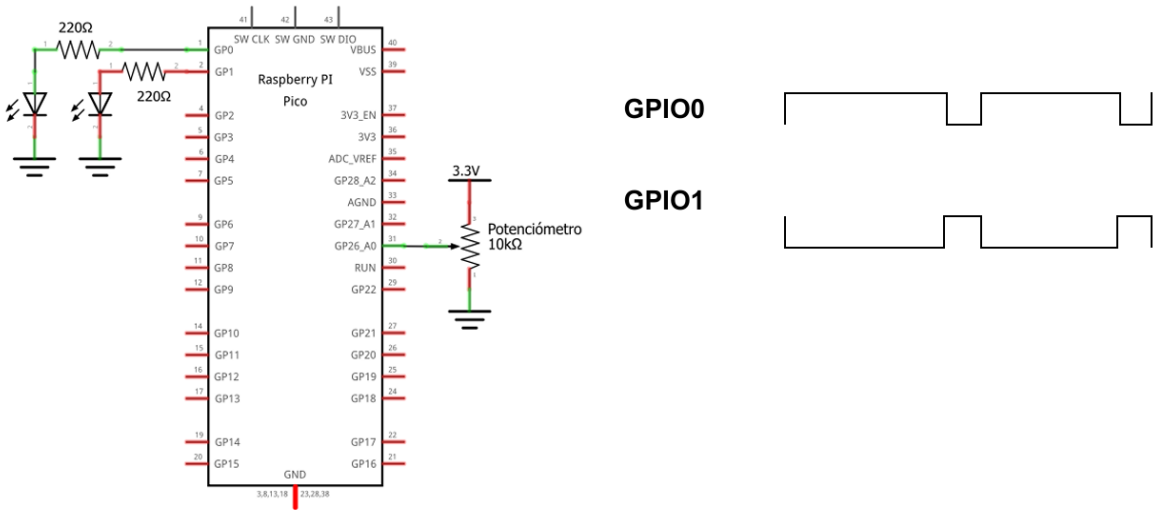


Figura 11: Control PWM por medio de entrada analógica; actividad 7

2.7.1. Pseudo Código

Algoritmo 7: Control PWM con potenciómetro

Inicializar ADC en GPIO26 (potenciómetro);
Inicializar GPIO0 como PWM;
Inicializar GPIO1 como PWM;
Configurar frecuencia de 1000 Hz en ambos pines;
Mientras el programa esté en ejecución hacer;
    Leer valor del potenciómetro;
    PWM GPIO0 = valor leído;
    PWM GPIO1 = 65535 - valor leído;
    Esperar 0.1 segundos;

2.7.2. Desarrollo

```
from machine import ADC, Pin, PWM
import time

pot = ADC(26)

pwm0 = PWM(Pin(0))
pwm1 = PWM(Pin(1))

pwm0.freq(1000)
pwm1.freq(1000)

while True:

```

```
    valor = pot.read_u16()

    pwm0.duty_u16(valor)
    pwm1.duty_u16(65535 - valor)

    time.sleep(0.1)
```

2.7.3. Análisis de resultados

El potenciómetro controla directamente el PWM. Al girarlo, un PWM aumenta mientras el otro disminuye, generando una señal inversa controlada manualmente.

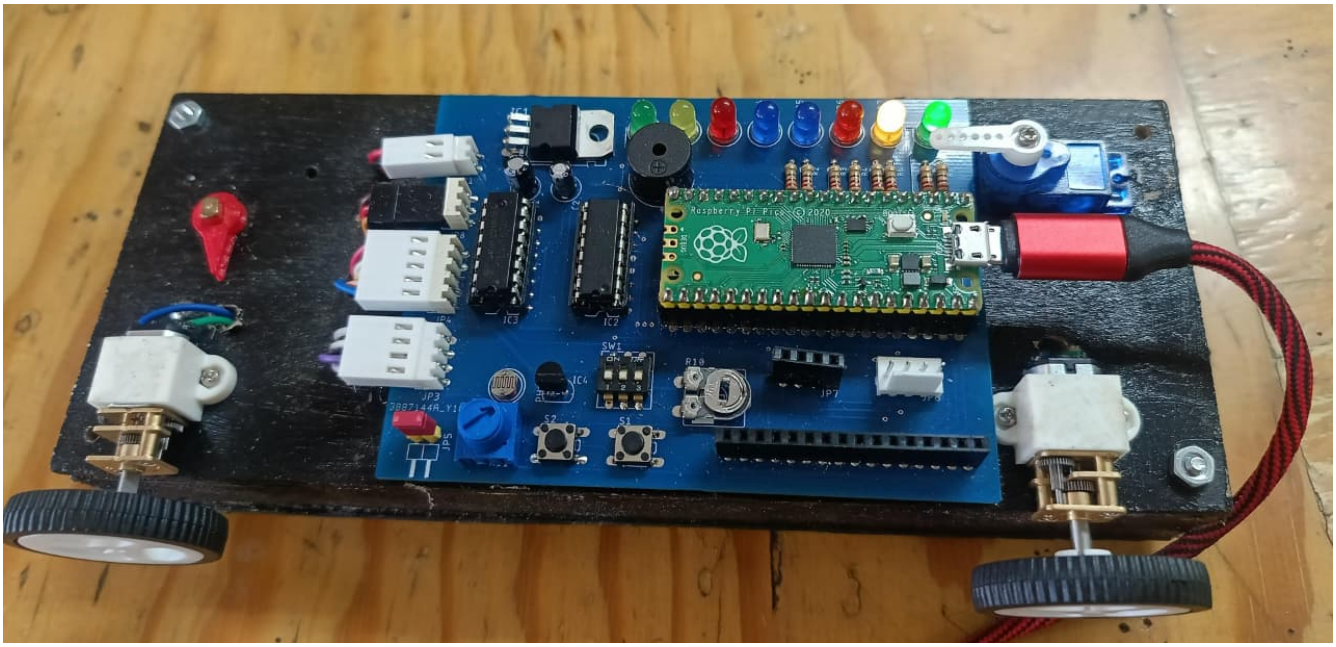


Figura 12

3. Conclusiones

El desarrollo de esta práctica se llevó a cabo gracias al tutorial de Raspberry Pi de la facultad [1] y a las clases del profesor Moisés [2].

3.1. Hernandez Acosta Mauricio Gabriel

Esta práctica me ayudó mucho a entender cómo un microcontrolador realmente interactúa con lo que pasa en el mundo exterior. A veces uno se acostumbra a trabajar únicamente con señales digitales de ceros y unos, por lo que ver cómo el ADC traduce una variación continua de voltaje a valores numéricos en Thonny me pareció bastante intuitivo y revelador. Me gustó mucho el ejercicio del PWM con desvanecimiento, ya que pude comprobar en físico cómo al cambiar el ciclo de trabajo en fracciones de segundo la vista percibe un cambio gradual de brillo en el LED en lugar de parpadeos bruscos. En general, la práctica me dejó un aprendizaje muy claro sobre cómo leer sensores analógicos y cómo usar MicroPython para manipular periféricos de salida sin tantas complicaciones de bajo nivel.

3.2. Perez Olvera Alexis Abraham

Lo que más me gustó de esta práctica fue la parte de comparar los dos sensores de temperatura. Al principio pensé que el sensor interno del RP2040 sería suficiente para cualquier medición, pero al poner a prueba el TMP36 nos dimos cuenta de que el chip se calienta un poco por su propia operación y eso altera las lecturas, algo que en la teoría se menciona pero que no te queda del todo claro hasta que lo ves corriendo en la consola. También me pareció muy satisfactorio el último ejercicio, donde conectamos el potenciómetro para gobernar las dos salidas PWM complementarias; girar la perilla y ver cómo la intensidad de un LED subía mientras el otro se apagaba de forma suave y sin retrasos demostró lo útil que es combinar una entrada analógica con salidas moduladas para tareas de control en tiempo real.

3.3. Sierra Garcia Mariana

Personalmente encontré muy entretenida y formativa esta práctica, sobre todo al trabajar con la fotoresistencia y armar el divisor de voltaje. Fue muy gratificante tapar el sensor con la mano y ver cómo el programa detectaba la caída de luz en tiempo real para activar la salida en el GPIO7, funcionando justo como el encendido automático de las luces públicas. Además, programar las señales PWM inversas en los pines 0 y 1 me ayudó a entender mejor la lógica detrás de los actuadores complementarios, como los que se usan para cambiar el sentido de giro de un motor o en controles de potencia balanceados. Siento que esta sesión nos dio herramientas muy prácticas que sin duda aplicaremos en proyectos más avanzados del laboratorio.

Referencias

[1] R. Anaya García et al., *Manual de Aplicaciones Raspberry Pi*, Manual de recursos y aplicaciones para plataforma Raspberry Pi, Universidad Nacional Autónoma de México, Facultad de Ingeniería, Laboratorio de Microcomputadoras, Ciudad Universitaria Coyoacán, México, 2026. dirección: https://odin.fi-b.unam.mx/micros/docs/Tutoriales%20Raspberry.pdf

[2] M. Meléndez Reyes, *Notas de clase: Práctica 01 - Introducción a las arquitecturas ARM empleando Raspberry Pi*, Clase presencial en laboratorio de microprocesadores, Facultad de Ingeniería, UNAM, Material proporcionado por el profesor durante la sesión de laboratorio, feb. de 2026.